

wfrest

peanutixx

2026 年 4 月 3 日

Contents

1 简介	1
2 安装	2
3 快速入门	2
4 定义路由	3
5 解析请求	5
5.1 解析路径	5
5.2 解析查询参数	5
5.3 解析请求头	7
5.4 解析请求主体	8
6 处理业务：完成各种各样的任务	10
6.1 上传文件	10
6.2 下载文件	11
6.3 静态资源服务器	13
6.4 SeriesHandler：集成 Workflow	14
6.5 MySQL 任务	15

1 简介

wfrest 是一个基于 C++ Workflow 引擎的高性能、异步、轻量级 **RESTful** 框架。它的核心设计目标是：一方面，利用 Workflow 引擎的高性能异步能力来处理高并发；另一方面，为开发者提供一套像 Python Flask 或 Go Gin 那样简洁易用的 API，从而极大地降低 C++网络服务的开发门槛。

要理解 **wfrest**，首先得了解它的基石 — C++ Workflow。这是搜狗公司开源的、非常轻量级的 C++并行计算和异步网络引擎。你可以把它想象成一个高度优化的“异步任务调度器”，能够高效地管理网络通信和计算任务。

wfrest 巧妙地封装了 Workflow 的复杂性，将 **Workflow** 底层的网络通信细节、异步回调等繁杂操作，都隐藏在了自己简洁的接口之下，让开发者能专注于业务逻辑，而无需过多关心底层 I/O 和并发模型的实现。

2 安装

wfrest 是一个标准的基于 CMake 构建的项目，我们可以通过以下步骤进行安装：

```
# 进入项目源代码目录
cd wfrest
```

```

# 创建并进入构建目录（用于存放编译中间文件和结果）
mkdir build && cd build
# 运行 CMake 配置项目，CMakeLists.txt 在上一级目录（..）
cmake ..
# 编译项目，生成可执行文件和库文件
make
# 将编译好的文件安装到系统目录（如 /usr/local/bin、/usr/local/lib）
sudo make install
# 更新动态链接库缓存，使系统能找到新安装的库文件
sudo ldconfig

```

安装成功后，`/usr/local/include` 会多一个文件夹 `wfrest`，里面放的是头文件。`/usr/local/lib` 目录下会生成 `libwfrest.a` 和 `libwfrest.so` 两个库文件。

3 快速入门

使用 `wfrest` 搭建服务器非常直观。下面是一个最基础的“Hello World”示例，它清晰地展示了 `wfrest` 框架的简洁性：

```

                                01_hello_wfrest.cc
1  #include <iostream>
2  #include <signal.h>
3  #include <wfrest/HttpServer.h>
4  #include <workflow/WFFacilities.h>
5
6  using namespace std;
7  using namespace wfrest;
8
9  // 用于优雅退出的全局等待组
10 static WFFacilities::WaitGroup waitGroup(1);
11
12 void sig_handler(int signum)
13 {
14     waitGroup.done();
15 }
16
17 int main()
18 {
19     // 1. 注册信号处理函数（按 Ctrl+C 退出）
20     signal(SIGINT, sig_handler);
21     // 2. 使用默认参数创建服务器，开箱即用
22     HttpServer server;
23     // 3. 定义路由和处理逻辑
24     server.GET("/hello", [](const HttpReq* req, HttpResp* resp) {
25         resp->String("world\n"); // 返回一个简单的字符串
26     });
27
28     server.POST("/echo", [](const HttpReq* req, HttpResp* resp) {
29         // 直接返回请求的主体，实现 echo 服务
30         resp->String(req->body());
31     });
32     // 3. 启动服务器，绑定通配符地址，监听 8888 端口

```

```

33     if (server.start(8888) == 0) {
34         waitGroup.wait(); // 主线程等待，直到收到退出信号
35         server.stop();
36     } else {
37         cerr << "Error: Server start FAILED!" << endl;
38         exit(1);
39     }
40 }

```

这个例子展示了如何用几十行代码创建一个支持 GET、POST 请求的 Web 服务，其简洁程度不亚于许多脚本语言框架。此外，wfrest 还提供了丰富的特性来满足各种开发需求。更多信息，请查阅 [wfrest GitHub 仓库](#)。

4 定义路由

简单来说，路由（Routing）就是一个桥梁，它负责将客户端的请求映射到服务器端的处理函数上。路由的作用就是根据请求方法和路径，找到并执行对应的代码块。请看下面示例：

```

                                02_register_routes.cc
1  #include <iostream>
2  #include <signal.h>
3  #include <wfrest/HttpServer.h>
4  #include <workflow/WFFacilities.h>
5
6  using namespace std;
7  using namespace wfrest;
8
9  static WFFacilities::WaitGroup waitGroup(1);
10
11 void sig_handler(int signum)
12 {
13     waitGroup.done();
14 }
15
16 int main()
17 {
18     // 1. 注册信号处理函数（按 Ctrl+C 退出）
19     signal(SIGINT, sig_handler);
20     // 2. 使用默认参数，创建 HttpServer，开箱即用
21     HttpServer server;
22     // 3. 设置路由（相当于在 process() 函数中设置一个分支）
23     // 静态路由：
24     server.GET("/user/xixi", [](const HttpReq* req, HttpResp* resp) {
25         resp->String("特别的爱给特别的你，我的寂寞逃不过你的眼睛\n");
26     });
27     // 参数路由：
28     // 此路由将匹配 /user/peanut，但不匹配 /user/xixi 或 /user/
29     server.GET("/user/{name}", [](const HttpReq* req, HttpResp* resp) {
30         // 获取路径参数
31         const string& name = req->param("name");
32         cout << "name: " << name << endl;
33         // 设置返回状态码

```

```

34     resp->set_status(HttpStatusOK); // 默认返回 HttpStatusOK: 200
35     resp->String("Hello " + name + "\n");
36 });
37 // 通配符路由:
38 server.GET("/wildcard/*", [](const HttpRequest* req, HttpResponse* resp) {
39     // 解析路径
40     cout << "match_path: " << req->match_path() << endl;
41     cout << "full_path: " << req->full_path() << endl;
42     cout << "current_path: " << req->current_path() << endl;
43 });
44 // 参数和通配符可以一起使用
45 server.GET("/user/{name}/*", [](const HttpRequest* req, HttpResponse* resp) {
46     string name = req->param("name");
47     cout << "name: " << name << endl;
48     cout << "match_path: " << req->match_path() << endl;
49     cout << "full_path: " << req->full_path() << endl;
50     cout << "current_path: " << req->current_path() << endl;
51 });
52 // 4. 启动服务器
53 if (server.start(8888) == 0) {
54     server.list_routes(); // 打印所有注册的路由
55     waitGroup.wait();
56     server.stop();
57 } else {
58     cerr << "Error: Server start FAILED!" << endl;
59     exit(1);
60 }
61 return 0;
62 }

```

wfrest 支持三种模式的路由:

路由类型	匹配规则	示例	应用场景
静态路由	精确的字符串匹配	/users/list	用于匹配固定的、不变的 API 端点。
参数路由	使用{参数名} 捕获 URL 路径中的动态部分，参数值可通过 req->param("参数名") 获取。	/user/{id}	用于 RESTful API，例如根据用户 ID 获取用户信息。
通配符路由	*号可以匹配任意路径段，匹配的路径段可通过 req->match_path() 获取。	/static/*	通常用于处理静态文件或作为“捕获所有”的路由。

注意：路由解析

wfrest 会最先匹配静态路由，参数路由和通配符路由的匹配顺序官方文档未说明，我们不能做任何假定。

5 解析请求

5.1 解析路径

在 Section 4 中，我们已经看到 wfrest 是如何解析路径的了：

```

45     server.GET("/user/{name}/*", [](const HttpReq* req, HttpResp* resp) {
46         string name = req->param("name");
47         cout << "name: " << name << endl;
48         cout << "match_path: " << req->match_path() << endl;
49         cout << "full_path: " << req->full_path() << endl;
50         cout << "current_path: " << req->current_path() << endl;
51     });

```

总结如下：

- req->param(): 获取路径参数。
- req->match_path(): 获取*号匹配的路径段。
- req->full_path(): 获取定义路由时写入的路径模式。
- req->current_path(): 获取用户输入的路径。

5.2 解析查询参数

我们先来看下面这个示例程序：

```

1  #include <iostream>
2  #include <signal.h>
3  #include <wfrest/HttpServer.h>
4  #include <workflow/WFFacilities.h>
5
6  using namespace std;
7  using namespace wfrest;
8
9  static WFFacilities::WaitGroup waitGroup(1);
10
11 void sig_handler(int signum)
12 {
13     waitGroup.done();
14 }
15
16 int main()
17 {
18     // 1. 注册信号处理函数（按 Ctrl+C 退出）
19     signal(SIGINT, sig_handler);
20     // 2. 创建 HTTP 服务器
21     HttpServer server;
22     // 3. 定义路由
23     // URL: /all_queries?username=peanut&password=lovesixxi
24     server.GET("/all_queries", [](const HttpReq* req, HttpResp* resp) {
25         const map<string, string>& all_queries = req->query_list();
26         // const auto& all_queries = req->query_list(); // 熟悉之后可以这样使用
27         // 结构体绑定: C++17 特性
28         for (const auto& [name, value] : all_queries) {
29             cout << name << ": " << value << endl;
30         }

```

```

31     });
32
33     // URL: /query?username=peanut&passwd=lovesixi
34     server.GET("/query", [](const HttpReq* req, HttpResp* resp) {
35         const string& username = req->query("username");
36         const string& password = req->query("password");
37         // 没有 info 参数, 返回空字符串
38         const string& info = req->query("info");
39         assert(info == "" && "info doesn't equal \"\"");
40         // 没有 address 参数, 返回默认值 "china"
41         const string& address = req->default_query("address", "china");
42         cout << "username: " << username
43             << ", password: " << password
44             << ", info: " << info
45             << ", address: " << address << endl;
46     });
47
48     // URL: /has_query?username=peanut&password=
49     // 查询参数的值可以为空, 所以不能通过判断参数的值是否为空, 来判断参数是否存在
50     // 而应该通过 has_query() 来判断
51     server.GET("/has_query", [](const HttpReq* req, HttpResp* resp) {
52         cout << req->query("password") << endl;
53         if (req->has_query("password")) {
54             cout << "存在参数 password" << endl;
55         }
56         if (req->has_query("info")) {
57             cout << "存在参数 info" << endl;
58         }
59     });
60     // 4. 启动服务器
61     if (server.start(8888) == 0) {
62         server.list_routes();
63         waitGroup.wait();
64         server.stop();
65     } else {
66         cerr << "Error: server start FAILED!" << endl;
67         exit(1);
68     }
69     return 0;
70 }

```

总结如下:

- req->query_list(): 获取所有查询参数。
- req->query(name): 获取指定的查询参数, 如果没有, 返回空字符串。
- req->default_query(name, defaultValue): 获取指定的查询参数, 如果没有, 返回默认值 defaultValue。
- req->has_query(name): 判断指定的查询参数是否存在。

5.3 解析请求头

先来看下面的例子：

```
04_request_header.cc
1 #include <iostream>
2 #include <signal.h>
3 #include <wfrest/HttpServer.h>
4 #include <workflow/HttpUtil.h>
5 #include <workflow/WFFacilities.h>
6
7 using namespace std;
8 using namespace wfrest;
9 using namespace protocol;
10
11 static WFFacilities::WaitGroup waitGroup(1);
12
13 void sig_handler(int signum)
14 {
15     waitGroup.done();
16 }
17
18 int main()
19 {
20     signal(SIGINT, sig_handler);
21
22     HttpServer server;
23
24     server.POST("/all_headers", [](const HttpReq* req, HttpResp* resp) {
25         // wfrest 没有专门用来遍历请求头的方法
26         // 不过我们可以借助 workflow 中的 HttpHeadersCursor 来遍历
27         HttpHeadersCursor cursor(req);
28         string name;
29         string value;
30         while (cursor.next(name, value)) {
31             cout << name << ": " << value << endl;
32         }
33     });
34
35     server.POST("/header", [](const HttpReq* req, HttpResp* resp) {
36         // 获取请求头
37         const string& host = req->header("Host");
38         const string& contentType = req->header("Content-Type");
39         const string& xyz = req->header("xyz"); // 没有 xyz 请求头，会返回空字符串
40         cout << "Host: " << host << endl;
41         cout << "Content-Type: " << contentType << endl;
42         cout << "xyz: " << xyz << endl;
43     });
44
45     server.POST("/has_header", [](const HttpReq* req, HttpResp* resp) {
46         if (req->has_header("Host")) {
47             cout << "Host: " << req->header("Host") << endl;
```

```

48     }
49     if (req->has_header("xyz")) {
50         cout << "xyz: " << req->header("xyz") << endl;
51     }
52 });
53
54 if (server.start(8888) == 0) {
55     server.list_routes();
56     waitGroup.wait();
57     server.stop();
58 } else {
59     cerr << "Error: Server start FAILED!" << endl;
60     exit(1);
61 }
62 return 0;
63 }

```

总结如下：

- wfrest 没有专门遍历报文头部的方法，需要借助 Workflow 中的 `HttpHeaderCursor`。
- `req->header(name)`：获取指定的头部字段，如果没有，返回空字符串。
- `req->has_header(name)`：判断指定的头部字段是否存在。

5.4 解析请求主体

先来看下面的例子：

```

05_request_body.cc
1 #include <iostream>
2 #include <signal.h>
3 #include <wfrest/HttpServer.h>
4 #include <workflow/WFFacilities.h>
5
6 using namespace std;
7 using namespace wfrest;
8
9 static WFFacilities::WaitGroup waitGroup(1);
10
11 void sig_handler(int signum)
12 {
13     waitGroup.done();
14 }
15
16 int main()
17 {
18     signal(SIGINT, sig_handler);
19
20     HttpServer server;
21
22     server.POST("/request_body", [](const HttpReq* req, HttpResp* resp) {
23         // 只是单纯地获取请求体

```



```

24     // 不会解析 x-www-form-urlencoded 表单
25     // 不会解析 form-data 表单
26     string& body = req->body();
27     cout << body << endl;
28 });
29
30 // curl -v http://ip:port/form-urlencoded \
31 // -H "Content-Type: application/x-www-form-urlencoded" \
32 // -d 'user=admin&password=123456'
33 server.POST("/form-urlencoded", [](const HttpReq* req, HttpResp* resp) {
34     // 校验请求体的类型: application/x-www-form-urlencoded
35     if (req->content_type() != APPLICATION_URL_ENCODED) {
36         resp->set_status(HttpStatusBadRequest); // 响应状态码: 400 BadRequest
37         return;
38     }
39     // req->form_kv() 获取表单数据
40     map<string, string>& form = req->form_kv();
41     for (const auto& [key, value] : form) {
42         cout << key << ": " << value << endl;
43     }
44 });
45
46 // curl -X POST http://ip:port/form-data \
47 // -F "file=@/path/file" \
48 // -H "Content-Type: multipart/form-data"
49 server.POST("/form-data", [](const HttpReq* req, HttpResp* resp) {
50     // 校验请求体的类型: multipart/form-data
51     if (req->content_type() != MULTIPART_FORM_DATA) {
52         resp->set_status(HttpStatusBadRequest); // 响应状态码: 400 BadRequest
53         return;
54     }
55     /*
56      * https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST
57      * <key, <filename, content>>
58      * using Form = map<string, pair<string, string>>;
59      */
60     const Form& formData = req->form();
61     for (const auto& [key, file] : formData) {
62         cout << "key: " << key << endl;
63         cout << "filename: " << file.first << endl;
64         cout << "content: " << file.second << endl;
65     }
66 });
67
68 if (server.start(8888) == 0) {
69     server.list_routes();
70     waitGroup.wait();
71     server.stop();
72 } else {
73     cerr << "Error: Server start FAILED!" << endl;
74     exit(1);

```

```

75     }
76     return 0;
77 }

```

总结如下:

- req->body(): 获取请求体, 但是不会做进一步解析。
- req->form_kv(): 专门用于 application/x-www-form-urlencoded 类型的数据, 会将请求体解析为 map<string, string>。
- req->form(): 专门用于 multipart/form-data 类型的数据, 会将请求体解析为 Form=map<string, pair<string, string>>。

6 处理业务: 完成各种各样的任务

6.1 上传文件

请看下面示例:

```

                                06_upload_file.cc
1  #include <iostream>
2  #include <signal.h>
3  #include <wfrest/HttpServer.h>
4  #include <wfrest/PathUtil.h>
5  #include <workflow/WFFacilities.h>
6
7  using namespace std;
8  using namespace wfrest;
9
10 static WFFacilities::WaitGroup waitGroup(1);
11
12 void sig_handler(int signum)
13 {
14     waitGroup.done();
15 }
16
17 int main()
18 {
19     signal(SIGINT, sig_handler);
20
21     HttpServer server;
22
23     // curl -v -X POST "ip:port/upload-file" -F "file=@demo.txt;
24     // filename=../demo.txt" -H "Content-Type: multipart/form-data"
25     // 1. 获取文件的名称
26     // 2. 获取文件的内容
27     // 3. 保存文件
28     server.POST("/upload-file", [](const HttpReq* req, HttpResp* resp) {
29         // 校验请求体的类型: multipart/form-data
30         if (req->content_type() != MULTIPART_FORM_DATA) {
31             resp->set_status(HttpStatusBadRequest); // 400: BadRequest
32             return;
33         }
34         //                                     key          filename, content

```

```

35     // using Form = map<string, pair<string, string>>
36     const Form& form = req->form();
37
38     for (const auto& [key, file] : form) {
39         const string& filename = file.first;
40         const string& content = file.second;
41 #ifdef DEBUG
42         std::cout << filename << ": " << content << endl;
43 #endif
44         // filename 不应该被信任, 不应该被应用程序盲目使用。
45         // 参见 MDN 上的 Content-Disposition
46         // https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Content-Disposition#directives
47         // 应去除路径信息, 并应用服务器文件系统规则
48
49         // resp->Save(filename, std::move(content)); // Wrong: 有安全隐患
50         string basename = PathUtil::base(filename);
51         // resp->Save(basename, std::move(content));
52         resp->Save(basename, std::move(content), "Save " + basename + " Success\n");
53     }
54 });
55
56 if (server.start(8888) == 0) {
57     server.list_routes();
58     waitGroup.wait();
59     server.stop();
60 } else {
61     cerr << "Error: Server start FAILED!" << endl;
62     exit(1);
63 }
64 return 0;
65 }

```

总结如下:

- HTTP 协议是通过 multipart/form-data 类型来上传文件的。
- 注意: 用户指定的文件名是不应该被信任的, 必须使用 PathUtil::base(filename) 进行处理。
- resp->Save(path, content) 可以将文件内容保存到指定路径, 它封装了 Workflow 的 WFFileIOTask。
- resp->Save(path, content, msg): 保存成功后, 给用户返回消息 msg。

6.2 下载文件

请看下面示例:

```

1 #include <iostream>
2 #include <signal.h>
3 #include <wfrest/HttpServer.h>
4 #include <workflow/WFFacilities.h>
5
6 using namespace std;
7 using namespace wfrest;

```

```

8
9 static WFFacilities::WaitGroup waitGroup(1);
10
11 void sig_handler(int signum)
12 {
13     waitGroup.done();
14 }
15
16 int main()
17 {
18     signal(SIGINT, sig_handler);
19
20     HttpServer server;
21
22     server.GET("/download-file1", [](const HttpRequest* req, HttpResponse* resp) {
23         // 支持绝对路径, 但一般不要使用
24         resp->File("/home/peanut/a.txt");
25     });
26
27     server.GET("/download-file2", [](const HttpRequest* req, HttpResponse* resp) {
28         // 一般使用相对路径
29         resp->File("resources/a.txt");
30     });
31
32     server.GET("/download-file3", [](const HttpRequest* req, HttpResponse* resp) {
33         // 支持范围请求
34         resp->File("resources/a.txt", 6);
35     });
36
37     server.GET("/download-file4", [](const HttpRequest* req, HttpResponse* resp) {
38         // 支持范围请求
39         resp->File("resources/a.txt", 6, 11);
40     });
41
42     if (server.start(8888) == 0) {
43         server.list_routes();
44         waitGroup.wait();
45         server.stop();
46     } else {
47         cerr << "Error: cannot start server!\n";
48         exit(1);
49     }
50     return 0;
51 }

```

总结如下:

- `resp->File(path, ...)` 可以读取文件内容并返回给客户端, 它封装了 Workflow 的 `WFFileIOTask`。
- `path` 可以是绝对路径, 也可以是相对路径, 并且 `resp->File(path, ...)` 还支持范围请求。

IDEA: 静态资源服务器

如果我们能通过用户传入的路径得知他想要哪个文件，那么我们就可以用 `resp->File()` 轻易地实现一个静态资源服务器。

6.3 静态资源服务器

请看下面示例：

```
08_static_file_server.cc
1 #include <signal.h>
2 #include <wfrest/HttpServer.h>
3 #include <workflow/WFFacilities.h>
4
5 using namespace std;
6 using namespace wfrest;
7
8 static WFFacilities::WaitGroup waitGroup(1);
9
10 void sig_handler(int signum)
11 {
12     waitGroup.done();
13 }
14
15 int main()
16 {
17     signal(SIGINT, sig_handler); // 注册 Ctrl+C 信号处理
18
19     HttpServer server;
20
21     // 2. 设置路由
22     // server.GET("/static/*", [](const HttpReq* req, HttpResp* resp) {
23     //     string parent = "./www/static/";
24     //     string file = req->match_path();
25     //     resp->File(parent + file);
26     // });
27
28     // url.path --> filepath(目录, 文件)
29     server.Static("/static", "./www/static");
30     server.Static("/public", "./www");
31     server.Static("/", "./www/index.html");
32
33     if (server.start(8888) == 0) {
34         server.list_routes();
35         waitGroup.wait();
36         server.stop();
37     } else {
38         cerr << "Error: Server start FAILED!" << endl;
39         exit(1);
40     }
}
```

```
41     return 0;
42 }
```

总结如下:

- 使用 `server.Static(urlpath, filepath)` 可以轻松实现一个静态资源服务器, `filepath` 可以指向文件, 也可以指向目录。
- `server.Static(urlpath, filepath)` 底层是由 `server.GET(...)` 和 `resp->File(...)` 实现的。

6.4 SeriesHandler: 集成 Workflow

wfrest 支持两种类型的 Handler:

- `using Handler = std::function<void(const HttpReq*, HttpResp*)>;`
- `using SeriesHandler = std::function<void(const HttpReq*, HttpResp*, SeriesWork*)>;`

使用 `SeriesHandler` 可以轻易地与 `Workflow` 集成, 请看下面示例:

```
                                09_series_handler.cc
1  #include <iostream>
2  #include <signal.h>
3  #include <wfrest/HttpServer.h>
4  #include <workflow/WFFacilities.h>
5
6  using namespace std;
7  using namespace wfrest;
8
9  static WFFacilities::WaitGroup waitGroup(1);
10
11 void sig_handler(int signum)
12 {
13     waitGroup.done();
14 }
15
16 int main()
17 {
18     signal(SIGINT, sig_handler); // 注册 Ctrl+C 信号处理
19
20     HttpServer server;
21
22     server.GET("/series", [](const HttpReq* req, HttpResp* resp, SeriesWork* series) {
23         // 使用 Workflow 创建定时器任务
24         WFTimerTask* timerTask = WFTaskFactory::create_timer_task(3, 0, [](WFTimerTask*) {
25             cout << "定时器任务完成 (3 秒)" << endl;
26         });
27         // 往 SeriesWork 中添加 timerTask, 也可以添加各种各样的其它任务
28         series->push_back(timerTask);
29         // timerTask->start();
30         resp->String("Hello, SeriesTask!\n");
31     });
32
33     if (server.start(8888) == 0) {
```

```

34     server.list_routes();
35     waitGroup.wait();
36     server.stop();
37 } else {
38     cerr << "Error: Server start FAILED!" << endl;
39 }
40 return 0;
41 }

```

有了 `SeriesHandler`，我们就可以向处理当前请求的序列（`SeriesWork`）中添加各式各样的任务，完成各式各样的需求。

6.5 MySQL 任务

对于 MySQL 这种常见的任务，我们当然也可以使用 `SeriesHandler` 进行集成：

```

1 WfMySQLTask* task = WfTaskFactory::create_mysql_task(url, retry_max, callback);
2 MySQLRequest* req = task->get_req();
3 req->set_query(sql);
4 ...
5 series->push_back(task);

```

但 `wfrest` 为了方便程序员的使用，对这类常见的任务进行了封装。我们可以使用 `resp->MySQL(...)` 轻松完成上面的工作。

```

10_mysql_example.cc
1 #include <iostream>
2 #include <nlohmann/json.hpp>
3 #include <signal.h>
4 #include <wfrest/HttpServer.h>
5 #include <workflow/MySQLResult.h>
6 #include <workflow/WFFacilities.h>
7
8 using namespace std;
9 using namespace wfrest;
10 using namespace protocol;
11 using json = nlohmann::json;
12
13 static WFFacilities::WaitGroup waitGroup(1);
14
15 void sig_handler(int signum)
16 {
17     waitGroup.done();
18 }
19
20 int main()
21 {
22     signal(SIGINT, sig_handler);
23
24     HttpServer server;
25
26     server.GET("/mysql1", [](const HttpReq* req, HttpResp* resp) {

```

```

27     // 原理:
28     // 1. 创建 MySQL 任务,
29     // 2. push_back 到 SeriesWork 中,
30     // 3. 根据 MySQL 返回的结果集, 构建 JSON, 并返回给客户端
31     resp->MySQL("mysql://root:1234@localhost", "SHOW DATABASES");
32 });
33
34 // 执行多条 sql 语句
35 server.GET("/mysql2", [](const HttpReq* req, HttpResp* resp) {
36     string url = "mysql://root:1234@localhost/test";
37     string sql = "SHOW DATABASES; SELECT * FROM tbl_user";
38     resp->MySQL(url, sql);
39 });
40
41 /* 如果我们对默认的返回消息不满意, 也可以自定义
42 * using MySQLFunc = std::function<void(protocol::MySQLResultCursor* cursor)>;
43 */
44 server.GET("/mysql3", [](const HttpReq* req, HttpResp* resp) {
45     string url = "mysql://root:1234@localhost";
46     string sql = "SHOW DATABASES";
47     resp->MySQL(url, sql, [resp](MySQLResultCursor* cursor) {
48         // 自己构建返回消息
49         json result = json::array();
50         vector<MySQLCell> record;
51         while (cursor->fetch_row(record)) {
52             result.push_back(record[0].as_string());
53         }
54         resp->Json(result.dump());
55     });
56 });
57
58 if (server.start(8888) == 0) {
59     server.list_routes();
60     waitGroup.wait();
61     server.stop();
62 } else {
63     cerr << "Error: Server start FAILED!" << endl;
64     exit(1);
65 }
66 return 0;
67 }

```

总结:

- 使用 `resp->MySQL(url, sql)` 可以执行一条 SQL 语句, 也可以执行多条 SQL 语句。多条 SQL 语句之间用 ; 分隔。
- `resp->MySQL(url, sql)` 会以 JSON 格式返回所有的数据。如果我们对返回消息不满意, 可以使用 `resp->MySQL(url, sql, mysql_func)` 自定义返回消息。